

Project – High Level Design

on

Multi-Agent Automotive System

Course Name: Agentic AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	KANCHI TIWARI	EN22CS304032
2	ARYAN PAWAR	EN22CS304017
3	AMAN RAI	EN22CS304009
4	AAKRATI JAIN	EN22CS304001
5	PRATHAM BHAWAR	EN22CS304047
6	MOHAMMAD FARAZ ABBASI	EN22CS303033

Group Name: Group 10D11

Project Number: AAI-36

Industry Mentor Name:

University Mentor Name: Prof. Nishant Shrivastava

Academic Year: 2026

Table of Contents

1. Introduction.
 - 1.1. Scope of the document.
 - 1.2. Intended Audience
 - 1.3. System overview.
2. System Design.
 - 2.1. Application Design
 - 2.2. Process Flow.
 - 2.3. Information Flow.
 - 2.4. Components Design
 - 2.5. Key Design Considerations
 - 2.6. API Catalogue.
3. Data Design.
 - 3.1. Data Model
 - 3.2. Data Access Mechanism
 - 3.3. Data Retention Policies
 - 3.4. Data Migration
4. Interfaces
5. State and Session Management
6. Caching
7. Non-Functional Requirements
 - 7.1. Security Aspects
 - 7.2. Performance Aspects
8. References

1. Introduction

The **Multi-Agent Automotive System** is designed using an agentic AI architecture where multiple intelligent agents collaborate to perform specialized tasks. The system focuses on retrieving automotive information (such as car specifications) and presenting it in a structured, user-friendly format.

With the rapid growth of the automotive industry and the increasing demand for quick and reliable information about vehicles, users often need a system that can provide structured and easy-to-understand details such as specifications, features, performance metrics, and pricing. This system addresses that need by leveraging **Agentic AI**, where independent agents work together to achieve a common goal.

The system primarily consists of two core agents:

- **Researcher Agent:** Responsible for gathering relevant automotive data from external sources, APIs, or knowledge bases. It focuses on extracting accurate and up-to-date information about a given car model.
- **Writer Agent:** Responsible for processing the raw data collected by the researcher and transforming it into a well-structured, human-readable format suitable for display.

The interaction between these agents is managed through a coordinated workflow, ensuring smooth communication and task execution. The system uses modern AI frameworks such as **CrewAI / LangChain** along with Large Language Models (LLMs) to enable intelligent reasoning, content generation, and task delegation.

The user interacts with the system through a simple and intuitive interface (built using Streamlit), where they input the name of a car. The system then processes this input through the agents and returns a detailed and formatted output.

Overall, the Multi-Agent Automotive System demonstrates how distributed AI agents can collaborate effectively to solve real-world problems, making it a scalable and future-ready solution in the domain of intelligent information systems.

1.1 Scope of the Document

This document provides a detailed description of:

- System architecture
- Data handling mechanisms
- Design decisions
- Interfaces and workflows

It serves as a reference for developers, testers, and stakeholders.

1.2 Intended Audience

- Developers (Frontend & Backend)
- AI/ML Engineers
- Project Evaluators
- Testers
- Academic Reviewers

1.3 System Overview

The **Multi-Agent Automotive System** is an intelligent, AI-driven application that automates the process of retrieving and presenting automotive information through a collaborative multi-agent architecture. The system is designed to simulate how multiple specialized entities (agents) can work together efficiently to complete a complex task.

At a high level, the system accepts a car name as input from the user and processes it through a sequence of coordinated agents. Each agent performs a specific role, ensuring modularity and separation of concerns. The system is designed to simulate how multiple specialized entities (agents) can work together efficiently to complete a complex task

Working Overview

1. The user interacts with the system through a **Streamlit-based User Interface (UI)** and provides the name of a car.
2. The input is forwarded to the **Researcher Agent**, which is responsible for collecting relevant data such as:
 - Specifications (engine, mileage, power)
 - Features (safety, infotainment, etc.)
 - Pricing and performance details

3. The collected raw data is then passed to the **Writer Agent**.
4. The Writer Agent processes and formats the data into a structured and human-readable form.
5. The final output is displayed back to the user through the UI.

2. System Design

2.1 Application Design

The Multi-Agent Automotive System is designed using a modular and layered architecture that integrates multiple intelligent agents to perform specialized tasks. The application follows an agent-based design pattern, where each agent is responsible for a specific function, ensuring separation of concerns and improved system efficiency.

At a high level, the system accepts a **car name as input** from the user and processes it through a sequence of coordinated agents.

Architecture Overview

The application is structured into the following layers:

1. Presentation Layer (User Interface)

- Built using Streamlit
- Provides an interactive interface for users
- Accepts user input (car name)
- Displays the final processed output

2. Agent Layer

This is the core layer of the system where multiple agents collaborate:

➤ **Researcher Agent**

- Responsible for fetching automotive data
- Uses LLM and external tools/APIs
- Focuses on accuracy and completeness

➤ **Writer Agent**

- Processes the data received from the Researcher Agent
- Converts raw data into structured and readable format
- Ensures clarity and presentation quality

3. LLM Layer (AI Engine)

- Powered by Large Language Models Handles:
 - Natural language understanding
 - Reasoning and decision-making
 - Content generation
- Integrated using frameworks like CrewAI / LangChain

4. Data Layer

- Includes:
 - External APIs
 - Knowledge sources
 - (Optional) Vector database for embeddings
- Responsible for storing and retrieving relevant data

5. Design Approach

The system follows a pipeline-based workflow, where:

1. Input is received from the UI
2. Passed sequentially through agents
3. Each agent processes and forwards the output
4. Final result is displayed to the user

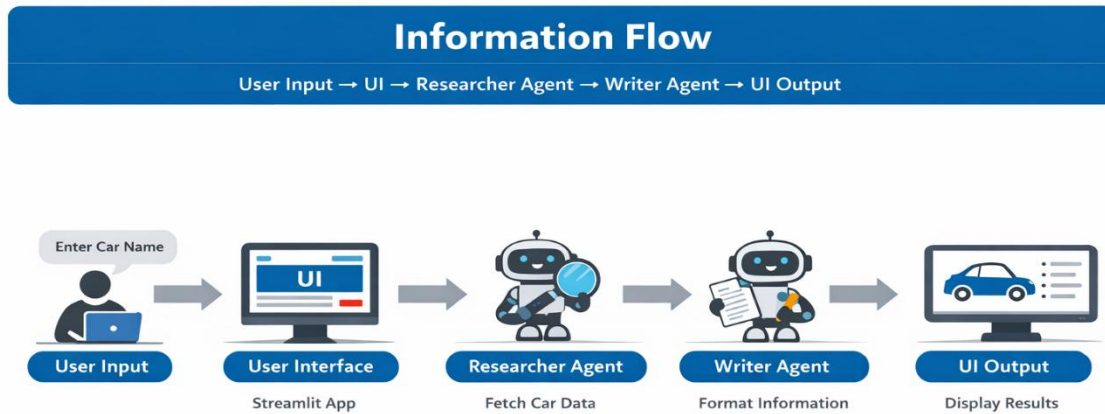
2.2 Process Flow

- **User enters car name** - The process begins when the user provides the name of a car through the input field in the application interface. This input acts as the primary query for the system and initiates the workflow.
- **Request sent to Researcher Agent** - Once the input is received, the system forwards the request to the Researcher Agent. This agent is responsible for understanding the query and preparing to fetch relevant automotive information.
- **Researcher gathers car specifications** - The Researcher Agent collects detailed information about the given car model from available sources such as APIs, datasets, or LLM knowledge. It retrieves key details including engine specifications, mileage, features, performance metrics, and pricing.
- **Data passed to Writer Agent** - The Writer Agent processes the received raw data and converts it into a structured, readable, and user-friendly format. It organizes the information into proper sections, improves clarity, and ensures the output is easy to understand.
- **Writer formats content** - After collecting the required information, the Researcher Agent sends the raw and unstructured data to the **Writer Agent** for further processing. This transfer ensures separation of data collection and content generation tasks.
- **Final output displayed on UI** - The formatted response is then sent back to the User Interface (Streamlit). The system displays the final structured information to the user in a clean and readable format, completing the workflow.

Information Flow



2.3 Information Flow



Step-by-Step Information Flow

1. User Input

- The process begins when the user enters a **car name** into the system.
- This input acts as the primary query for the entire workflow.

2. User Interface (Streamlit App)

- The input is received by the **UI layer** built using Streamlit.
- The UI acts as a bridge between the user and backend agents.

3. Researcher Agent

- The query is forwarded to the **Researcher Agent**.
- This agent collects relevant automotive data such as:
 - Engine specifications
 - Features
 - Performance details
 - Pricing information
- Data is gathered from APIs, datasets, or knowledge sources.

4. **Writer Agent**

- The collected raw data is passed to the **Writer Agent**.
- This agent processes and formats the information into a structured, readable format.
- It ensures clarity, consistency, and proper presentation.

5. **UI Output**

- The final formatted response is sent back to the UI.
- The system displays the processed information to the user in a clean and structured manner.

6. **End of Flow**

- The process completes once the output is shown.
- The system waits for the next user query.

2.4 Components Design

1. **UI Component**

- Built using Streamlit
- Accepts user input
- Displays results

2. **Researcher Agent**

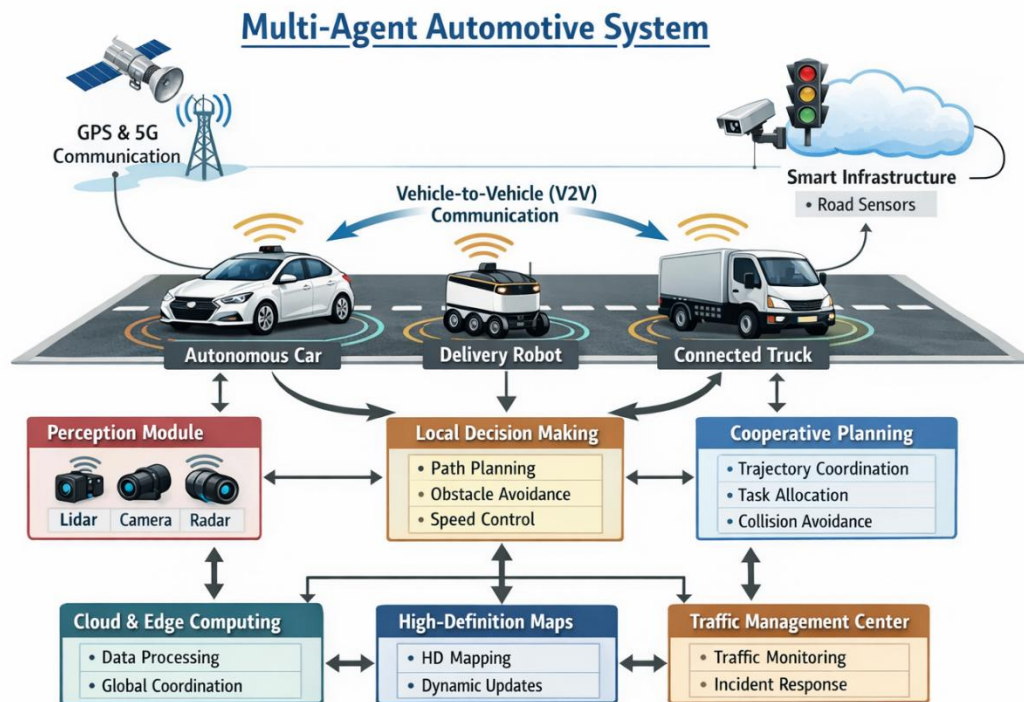
- Fetches car data
- Uses LLM + tools

3. **Writer Agent**

- Converts raw data into readable format

4. **LLM Integration**

- Provides reasoning and content generation



2.5 Key Design Considerations

- Scalability (adding more agents)
- Modularity
- Low latency
- Easy integration with APIs
- Maintainability

2.6 API Catalogue

All APIs follow RESTful standards.

API Name	Description
LLM API	Used for generating responses

Search API Fetches automotive data

Internal Agent API Communication between agents

3. Data Design

3.1 Data Model

Main Entities:

Vehicle

Data includes:

- Car Name
- Specifications (engine, mileage, price)
- Features
- Performance data

3.2 Data Access Mechanism

Data is accessed using:

- API calls
- LLM-based retrieval
- Optional vector database (for embeddings)

3.3 Data Retention Policies

- Temporary storage for session
- No long-term storage unless required
- Logs maintained for debugging

3.4 Data Migration

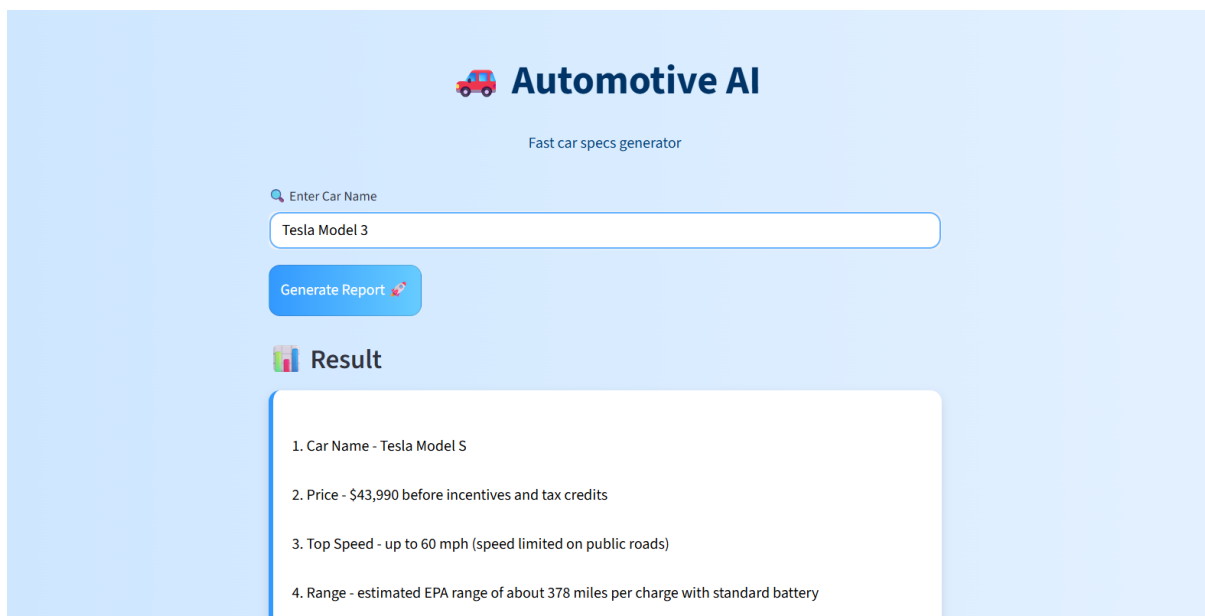
- Not applicable (stateless system)
- Can be extended with database in future

4. Interfaces

The **Multi-Agent Automotive System** provides well-defined interfaces to enable smooth interaction between the user, agents, and backend components. These interfaces ensure proper communication, data exchange, and system usability.

4.1 User Interface (UI)

- Built using **Streamlit**
- Provides a simple web-based interface for user interaction
- Accepts input in the form of **car name**
- Displays processed and formatted results
- Offers real-time feedback during agent execution (e.g., loading status)



The screenshot shows a web application titled "Automotive AI" with the subtitle "Fast car specs generator". It features a search bar labeled "Enter Car Name" containing the text "Tesla Model 3". Below the search bar is a blue button labeled "Generate Report". Underneath the button is a section titled "Result" which displays a list of specifications for the Tesla Model S:

- 1. Car Name - Tesla Model S
- 2. Price - \$43,990 before incentives and tax credits
- 3. Top Speed - up to 60 mph (speed limited on public roads)
- 4. Range - estimated EPA range of about 378 miles per charge with standard battery

4.2 Agent Interface

- Internal communication interface between **Researcher Agent** and **Writer Agent**
- Ensures structured data transfer between agents
- Uses LLM-based message passing and task delegation
- Maintains separation of responsibilities between agents

4.3 External API Interface

- Connects the system with external data sources (if used)
- Retrieves automotive data such as specifications, features, and pricing
- Communicates using REST APIs or LLM tool integrations

4.4 System Interface

- Connects UI, agents, and LLM backend
- Manages the execution flow of tasks
- Ensures proper request-response handling across components

5. State and Session Management

State Management

- The system does not permanently store user data
- Each request is treated independently (stateless processing)
- Temporary state is maintained only during execution of agents
- Intermediate data (like researcher output) exists only in runtime memory

Session Management

- Managed using **Streamlit session state**
- Helps maintain continuity during a single user session
- Stores temporary values

Session Lifecycle

1. User enters car name
2. Session state stores input
3. Agents process the request
4. Output is displayed
5. Session resets or waits for next input

Key Characteristics

- Stateless backend design (improves scalability)
- Temporary session storage only in UI layer
- No long-term data persistence
- Fast and lightweight execution

6. Caching

- Frequently requested data can be cached
- Reduces API calls and improves performance
- Example: same car search results

7. Non-Functional Requirements

7.1 Security Aspects

- API key protection
- Secure communication (HTTPS)
- Input validation

7.2 Performance Aspects

- Fast response time (<5 seconds)
- Efficient agent communication
- Minimal API latency

8. References

- CrewAI Documentation
- LangChain Documentation
- Streamlit Documentation
- OpenAI / LLM API Docs